AI-Driven Closed-Loop Precision Spraying for Real-Time Application Optimization

Team **CoverageVision AI** | Emery Brewer (remote) · Junjie Ao · Kinjalk Parth

Summary. Sprayers commit to nozzle, pressure, and speed settings before they can observe how weather and machine conditions will change the result, and standard quality checks with water-sensitive paper (WSP) report problems only after the sprayer has left the area. We propose a closed-loop system that predicts coverage and drift risk from pre-spray conditions, selects the setting that meets a coverage requirement at the lowest predicted drift risk and volume, measures the coverage actually achieved from WSP images with computer vision, and feeds the prediction error back into subsequent decisions. Without access to spray trials this semester, we build and evaluate the full loop in a physics-informed simulation with public WSP image data, and we state what that evaluation can and cannot show.

1 Problem Definition and Relevance

Spray performance depends on conditions that change during an application. Wind speed and direction, temperature, relative humidity, travel speed, boom height, nozzle type, pressure, and flow rate jointly determine droplet size, deposition, coverage uniformity, and off-target movement [1]. Sprayers, however, typically run on settings fixed before the pass. Rate controllers hold the application rate at its setpoint as travel speed changes, but the standard control loop does not measure the outcome that matters: the spray deposited on the target. Deposition is checked afterward, with water-sensitive paper (WSP) cards read by tools such as DepositScan, SnapCard, and DropLeaf [2–4]. The cards must be placed, retrieved, and imaged, so by the time a problem is detected the sprayer has moved beyond the area it affects.

Errors in either direction are costly. Under-coverage risks poor efficacy and repeat applications, over-application wastes product, and off-target movement damages neighboring crops. In 2017, off-target movement of dicamba injured an estimated 3.6 million acres of U.S. soybean and produced 2,708 state injury investigations [5]. Investigators identified physical drift as one of several pathways, alongside temperature inversions, volatility, tank contamination, and misuse [6]. Physical drift is the pathway that nozzle, pressure, speed, and boom-height decisions control directly [1].

We propose to close the gap between prediction and verification. Before each field segment, a model predicts coverage and drift risk from current conditions, and a selector chooses the setting that meets the coverage requirement at the lowest predicted drift risk and volume. After spraying, computer vision measures the coverage actually achieved, and the difference between measured and predicted coverage corrects the decisions for the next segments. Every verified segment also becomes a training record linking conditions, settings, and outcomes, so the system improves as it operates.

Scope. The team has no sprayer or field access this semester. We therefore build and evaluate the full loop in a physics-informed simulation and validate the vision component on public WSP images. WSP imaging stands in for the on-machine deposition sensing a deployed system would need; capacitance-based deposition sensors already report field deposition within 10% of a tracer reference [7].

2 Objectives and System Overview

The project has four objectives. Each has a lead, and all three members integrate the result (Section 5).

- **O1 Simulate.** Build a physics-informed simulator and dataset that map environmental, machine, and spray-system conditions to coverage, deposition, and drift. *Leads: Emery (relationships, scenarios) and Junjie (implementation).*
- **O2 Predict and select.** Predict spray performance from pre-spray conditions, and select the setting that meets

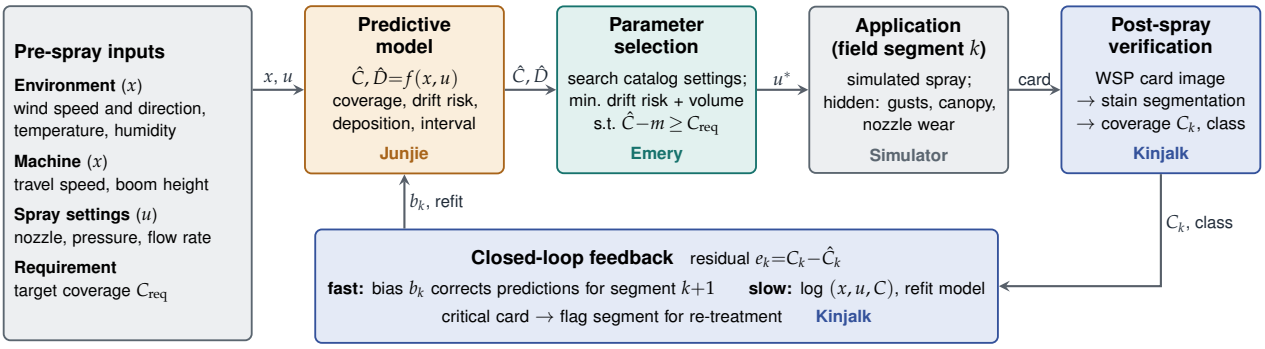


Figure 1. CoverageVision closed loop. The model predicts coverage $\hat{C}$ and drift risk $\hat{D}$ for candidate settings; the selector picks the setting u^* that meets C_{req} with margin m at the lowest predicted drift risk and volume; vision measures the coverage C_k achieved on a WSP card; the residual corrects the next segment (fast loop) and retrains the model (slow loop). Colors mark leads: ▢ Emery Brewer, ▢ Junjie Ao, ▢ Kinjalk Parth, ▢ shared simulator (Emery and Junjie).

a coverage requirement at the lowest predicted drift risk and application volume. *Leads: Junjie (model) and Emery (selection).*

O3 **Verify.** Measure coverage, stain density, and stain size from WSP card images, and classify each card against the requirement. *Lead: Kinjalk.*

O4 **Close the loop.** Use the gap between measured and predicted coverage to correct predictions and settings for subsequent segments, and quantify the benefit over open-loop operation. *Lead: Kinjalk, with all.*

Figure 1 shows how the components connect. We write x for conditions the controller cannot set (environment and machine state), u for the spray settings it chooses, $\hat{C}$ and $\hat{D}$ for predicted coverage and drift risk, and C for measured coverage. A field pass is divided into segments $k = 1, 2, \ldots$; the controller chooses one setting per segment, and each segment yields one verification card.

3 Expected Outcomes and Success Criteria

We fix quantitative targets before running experiments, so the final report can state which were met and why any were missed. Coverage classes follow the team's working thresholds, *acceptable* $\geq 30\%$, *marginal* 15–30%, and *critical* $< 15\%$. They remain configurable, because the appropriate coverage depends on the product and its target. A missed target is still a result: we will report every criterion with its measured value, including misses and their causes.

Table 1. Expected outcomes and success criteria (pp = percentage points of coverage).

Outcome	Success criterion (target)	Lead
End-to-end demonstration	A simulated field pass runs from prediction to correction without manual steps, and every decision is logged and visualized.	All
Simulator and dataset	Outputs follow the encoded trends (coverage rises with volume; drift rises with wind speed, release height, and fine-droplet share) across the M1 operating ranges.	Emery, Junjie
Predictive model	Coverage MAE ≤ 3 pp on held-out conditions and lower than a linear baseline's; error reported on held-out condition regimes; sensitivity checks pass.	Junjie
Parameter selection	With no injected mismatch, selected settings meet C_{req} in $\geq 90\%$ of scenarios at lower drift risk than the best fixed setting with the same success rate.	Emery
Post-spray verification	Coverage MAE ≤ 2 pp on synthetic and ≤ 5 pp on real cards (Section 4.3); three-class agreement $\geq 90\%$ on synthetic cards; ≤ 2 s per card on a laptop CPU.	Kinjalk
Closed loop	Under injected mismatch, recovers $\geq 50\%$ of the open-loop-to-oracle gap in requirement-met segments, using $\leq 10\%$ more volume than the oracle (20 seeds, 95% CI).	Kinjalk, all

4 Technical Approach and Feasibility

Each component uses established methods; the technical risk lies in integrating them into a loop whose behavior we can measure. The plan therefore freezes interfaces first and front-loads integration (Section 6).

4.1 Data and simulation • Emery • Junjie

Data tiers. We use public data where it exists and simulation where it does not. Nozzle manufacturer catalogs give flow rate and droplet-size class versus pressure for each nozzle, using the ASABE S572.3 categories [8]. Published field studies give the direction and approximate size of weather and machine effects on drift and deposition [1]. For imaging, Simões et al. release annotated WSP data, including 300 synthetic cards with per-droplet polygon labels (CC BY 4.0); their real evaluation set contains 2 manually annotated cards with 2,754 droplets [9].

Simulator. The simulator composes standard application-technology relationships. Application volume follows the boom-sprayer calibration equation, and nozzle flow scales with the square root of pressure:

$$\mathrm{GPA} = \frac{5940 \cdot \mathrm{GPM}}{\mathrm{MPH} \cdot W}, \qquad \frac{\mathrm{GPM}_2}{\mathrm{GPM}_1} = \sqrt{\frac{P_2}{P_1}}, \tag{1}$$

where W is the nozzle spacing in inches [10]. Each nozzle–pressure pair sets a droplet-size distribution (log-normal or Rosin–Rammler, parameterized by volume median diameter), following earlier WSP simulation studies [11]. The share of emitted volume lost to drift, which defines drift risk D, grows with wind speed, release height, and the fine-droplet fraction, and evaporation grows with temperature and falling humidity [1]. We fit these response curves to published trends and do not claim a validated drift model. Deposited droplets become stains through a spread factor and land at random positions, so stain overlap, and with it the non-linear relationship between deposit and coverage, emerges from the geometry [11].

Controlled mismatch. The simulator also contains factors the predictive model never observes: canopy interception, wind gusts, and nozzle wear. They create a known, adjustable gap between prediction and outcome, which is the error the feedback loop must detect and correct.

4.2 Predictive model and parameter selection • Junjie • Emery

Model. The model maps conditions x and a candidate setting u to predicted coverage, deposition, and drift risk. Inputs are wind speed and direction, temperature, relative humidity, travel speed, boom height, nozzle type, pressure, and flow rate. We start from a linear baseline, move to tree ensembles (random forest, gradient boosting), and add a small neural network only if the data support it. Evaluation uses a held-out test set (MAE, RMSE), a grouped split that withholds entire condition regimes to expose extrapolation error, and one-at-a-time sweeps that check whether predictions move in the physically expected direction. Where feasible, the model also outputs a prediction interval, which sets the selector's safety margin.

Selection. Parameter selection is a small constrained search:

$$u^* = \arg\min_{u \in \mathcal{U}} \; \lambda\, \hat{D}(x,u) + (1-\lambda)\, V(u) \quad \text{s.t.} \quad \hat{C}(x,u) - m \;\geq\; C_{\mathrm{req}}, \tag{2}$$

where $\mathcal{U}$ is the set of catalog settings (nozzle, pressure, flow rate), V is the application volume, both objective terms are normalized, and m is the margin. Because $\mathcal{U}$ holds a few hundred candidates, exhaustive evaluation with a tree ensemble takes milliseconds, fast enough to rerun for every field segment. If no setting satisfies the constraint, the selector returns the highest-coverage setting and flags the segment.

4.3 Post-spray verification • Kinjalk

Task. The verification module converts a WSP card image into coverage (percent of card area stained), stain density (stains per cm^2), and a stain-size distribution, then assigns the card to a coverage class. It is the component that makes the loop measurable, so it gets a classical baseline, a learned model, and an explicit error budget.

Training data. A synthetic renderer turns simulator output into card images with exact ground truth. Droplets sampled from the simulated size distribution become stains through the spread factor, overlap where they land, and are rendered with realistic stain colors plus random illumination gradients, blur, sensor noise, JPEG

compression, and perspective distortion. Unlike the public synthetic set, the renderer ties every card to the droplet spectrum and deposit that produced it, so image-level errors can be traced to spray-level causes.

Methods. Baseline B0 is a classical pipeline (color-space conversion, Otsu thresholding, contour analysis), the approach behind established WSP tools [2, 9]. Model B1 is a U-Net with a pretrained lightweight encoder; U-Net outperformed SegNet, PSPNet, and DeepLabv3+ on a dedicated WSP dataset [12]. We compute coverage from the semantic mask and treat stain counts as a secondary output from distance-transform watershed, because separating touching stains remains the weak point of current methods: a concave-point method validated on 553 field cards reports 77.8% segmentation accuracy, for cards below 25% coverage [13]. Cellpose, which detected droplets on real cards at 96.18% precision without fine-tuning [9], is our zero-shot alternative for counts.

Evaluation. We test on held-out cards from our renderer, on the Simões et al. synthetic set as an out-of-generator check, and on tiles of their annotated real cards. Metrics are coverage error in percentage points, stain-mask IoU, count error, three-class agreement, and runtime. We stratify errors by coverage band and stain size, because WSP physics bounds accuracy at both ends. Cards do not register droplets below about 50 µm and lose accuracy at high coverage [9], where stain overlap grows steeply [11]; pixel-based analysis also loses accuracy as spots shrink [2].

4.4 Closed-loop feedback • Kinjalk

The feedback module turns verification into control. At each verification point k, it computes the residual between measured and predicted coverage and updates a bias estimate:

$$e_k = C_k - \hat{C}_k, \qquad b_{k+1} = (1-\alpha)\, b_k + \alpha\, e_k, \qquad \hat{C}'_{k+1}(x,u) = \hat{C}(x,u) + b_{k+1}. \tag{3}$$

Fast loop (within a pass). The bias shifts predictions for the next segments, and the selector reruns on the corrected predictions. Corrections are clipped, wait for a minimum number of cards before acting, and are tracked per condition regime, so one gusty segment cannot retune the whole field. A card classified as critical also flags its segment for re-treatment, because no correction can fix an area the sprayer has already passed.

Slow loop (between passes). Each verified record (x, u, C) joins the training set, the model is refit, and we track whether residual variance falls over successive passes.

Evaluation. We compare open loop (model only), fast loop, fast plus slow loop, and an oracle selector that queries the true simulator, on simulated passes with time-varying wind and injected mismatch of increasing size. Metrics are the fraction of segments meeting C_{req}, mean coverage shortfall, applied volume, and drift risk, each reported as mean $\pm$ 95% confidence interval over 20 seeds. Because the simulator is ours, these experiments test whether the control logic corrects a known mismatch; they do not measure field efficacy, and the report will say so.

4.5 Software, compute, and integration

All code is Python in one GitHub repository: NumPy and SciPy for the simulator, scikit-learn and gradient-boosting libraries for tabular models, and PyTorch and OpenCV for vision. Modules exchange versioned records (conditions, settings, measurements), so each member develops against mock data before integration, and an end-to-end test runs the full loop before every merge. Tabular models train on a laptop CPU, and the vision models fit on a single GPU. The project needs no sprayer, field site, or laboratory equipment.

5 Team Responsibilities, Deliverables, and Collaboration

Each member leads one workstream from design to evaluation and reviews the others' code; Table 2 lists responsibilities and deliverables.

Collaboration tools. Microsoft Teams hosts the team channel, shared project documents, and biweekly meetings. Code lives in a shared GitHub repository with pull-request review.

Remote participation. Emery participates remotely, and the plan gives that role full weight. Emery's workstream needs no campus, lab, or field access, and it sits on the critical path: the M1 specification gates every other module, and the selector is one of the loop's four stages. Three mechanisms keep contributions balanced and visible. First, interfaces are frozen and versioned at M1, so remote and on-campus members integrate through code rather than in-person coordination. Second, every module merges only after review by another member, so each member

reads and tests teammates' work. Third, before each biweekly meeting every member posts a short written update to Teams, and decisions go into a shared log, so no one depends on having attended. Each member presents their own module at the final presentation.

Table 2. Responsibilities and deliverables by member. Milestones M1–M5 are defined in Table 3.

Member and skills	Responsibilities	Deliverables
Emery Brewer (remote) Systems engineering, application technology, agronomy	System specification (variables, units, operating ranges, coverage requirement and thresholds); application-technology relationships for the simulator; field-pass scenarios; parameter-selection module; agronomic interpretation of results.	System, interface, and simulator-relationship specification (M1); field-pass scenario library (M2); parameter-selection module with tests (M3); domain-validation section of the final report (M5).
Junjie Ao Machine learning, data analysis, predictive modeling, software development	Data pipeline for public, published, and simulated data; simulator implementation; predictive model development and evaluation; model interface to selection and feedback.	Prepared dataset for modeling (M2); predictive ML model and code (M3); model evaluation and validation results (M3–M4); model integrated into the final demonstration (M5).
Kinjalk Parth Computer vision, deep learning, segmentation of fine plant structures	Synthetic WSP renderer; verification module (classical baseline and learned model); feedback and recalibration logic; closed-loop experiments; end-to-end loop harness.	WSP renderer and card datasets (M2); verification module with synthetic and real-card evaluation (M3); feedback module and closed-loop results (M4); integrated demonstration (M5).

6 Milestones and Dependencies

Table 3. Milestones. Dates follow the Fall 2026 calendar (Thanksgiving break Nov 21–29; instruction ends Dec 9) and will move to match posted course deadlines.

	Date	Milestone and exit criterion	Lead
M1	Fri, Oct 2	**Specification freeze.** Variables, units, operating ranges, coverage metric and thresholds, and record schemas agreed; repository and public data in place.	Emery
M2	Fri, Oct 16	**Baselines.** Simulator v1 and dataset v1; baseline predictive model; scenario library; WSP renderer v1 with the classical verification baseline evaluated.	All
M3	Fri, Oct 30	**Components.** Model v2 with error analysis; selector running on the model; learned verification model evaluated on synthetic and real cards; feedback module unit-tested.	All
M4	Fri, Nov 20	**Integrated loop.** Full loop runs end to end; open-loop, closed-loop, and oracle experiments complete.	Kinjalk
M5	Wed, Dec 9	**Final deliverables.** Demonstration, report, and presentation.	All

Critical path. The simulator feeds both the predictive model and the card renderer, so simulator v1 at M2 is the first hard dependency. The feedback module needs both the model and the verification module, so a slip at M3 compresses M4. We protect M4 by building the loop harness against mock modules during M2–M3, which starts integration testing before the components are final.

Dependencies. *Data:* nozzle catalogs (flow and droplet class versus pressure), published drift and deposition trends [1], and annotated WSP images [9]; all are public and available now. *Software:* Python, NumPy/SciPy, scikit-learn, XGBoost or LightGBM, PyTorch, OpenCV, GitHub, and Microsoft Teams; all are open source or university-provided. *Simulation:* the spray simulator and WSP card renderer, built by the team by M2. *Hardware:* laptops and one GPU for vision training (lab workstation or Colab). *Field resources:* none; spray trials are out of scope this semester.

7 Risks and Mitigations

The largest risks sit in the vision module's transfer to real cards and in evaluating the loop inside our own simulator. Table 4 pairs each risk with its mitigation.

Table 4. Main risks and planned mitigations.

Risk	Mitigation
Sim-to-real gap in vision. Detectors trained on synthetic cards lose precision on real ones (YOLOv8: 57.06% synthetic vs. 40.81% real [9]).	Derive coverage from semantic masks, which need no instance detection; train with photometric and geometric degradations; keep B0 as a fallback; report real-card error separately.
Small real test set. Only 2 annotated real cards (2,754 droplets) are reported [9].	Report real-card results with their sample size as a sanity check, not a benchmark; if the annotations are not public, request them from the authors or report synthetic-only results and state the gap.
Circular evaluation. The loop is tested inside the team's own simulator.	Inject mismatch the model cannot observe, vary its size, compare against an oracle, and state the limitation.
WSP physical limits. Small droplets go unrecorded and accuracy drops at high coverage [9], where practical spread factors exceed datasheet values [14].	Stratify errors by coverage band; flag saturated cards instead of reporting a precise coverage; sample spread factors above datasheet values in the renderer.
Integration slip across three modules and two locations.	Interface freeze at M1, mock modules, an end-to-end test before every merge, and written updates before each meeting.

References

[1] A. S. Felsot, J. B. Unsworth, J. B. H. J. Linders, G. Roberts, D. Rautman, C. Harris, and E. Carazo. Agrochemical spray drift; assessment and mitigation—A review. *Journal of Environmental Science and Health, Part B*, 46(1):1–23, 2011.

[2] H. Zhu, M. Salyani, and R. D. Fox. A portable scanning system for evaluation of spray deposit distribution. *Computers and Electronics in Agriculture*, 76:38–43, 2011. doi:10.1016/j.compag.2011.01.003.

[3] C. Nansen, J. C. Ferguson, J. Moore, L. Groves, R. Emery, N. Garel, and A. Hewitt. Optimizing pesticide spray coverage using a novel web and smartphone tool, SnapCard. *Agronomy for Sustainable Development*, 35:1075–1085, 2015. doi:10.1007/s13593-015-0309-y.

[4] B. B. Machado, G. Spadon, M. S. Arruda, W. N. Gonçalves, A. C. P. L. F. Carvalho, and J. F. Rodrigues-Jr. A smartphone application to measure the quality of pest control spraying machines via image analysis. In *Proc. 33rd ACM Symposium on Applied Computing (SAC)*, 2018. doi:10.1145/3167132.3167237.

[5] K. Bradley. A final report on dicamba-injured soybean acres. *Integrated Pest & Crop Management*, University of Missouri, October 30, 2017.

[6] U.S. Environmental Protection Agency. Dicamba overview update. Pesticide Program Dialogue Committee, November 1, 2017.

[7] L. Li, R. Zhang, L. Chen, T. Yi, G. Xu, D. Xue, et al. Development of sensor system for real-time measurement of droplet deposition of agricultural sprayers. *International Journal of Agricultural and Biological Engineering*, 14(5), 2021. doi:10.25165/j.ijabe.20211405.5528.

[8] ANSI/ASABE S572.3. *Spray Nozzle Classification by Droplet Spectra*. American Society of Agricultural and Biological Engineers, St. Joseph, MI, 2020.

[9] I. Simões, A. J. Sousa, A. Baltazar, and F. Santos. Spray quality assessment on water-sensitive paper comparing AI and classical computer vision methods. *Agriculture*, 15(3):261, 2025. doi:10.3390/agriculture15030261; data: doi:10.5281/zenodo.13995950.

[10] Kansas State University Research and Extension. *Droplet Size Calibration: A New Approach to Effective Spraying*. Publication MF2869.

[11] E. Cerruto, S. Failla, D. Longo, and G. Manetto. Simulation of water sensitive papers for spray analysis. *Agricultural Engineering International: CIGR Journal*, 18(4), 2016.

[12] Q. Gao, M. Sozzi, and F. Marinello. U-Net-based semantic segmentation and concave point detection for adhesive droplets on water-sensitive paper. University of Padova research record, 2025. hdl:11577/3569369.

[13] E. Gil and L. Xun. A novel methodology for water-sensitive papers analysis focusing on the segmentation of overlapping droplets to better characterize deposition pattern. *Crop Protection*, 176:106492, 2024. doi:10.1016/j.cropro.2023.106492.

[14] E. Cerruto, G. Manetto, D. Longo, S. Failla, and R. Papa. A model to estimate the spray deposit by simulated water sensitive papers. *Crop Protection*, 124:104861, 2019. doi:10.1016/j.cropro.2019.104861.